

A

Project Report on

Malicious URL Detection System

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

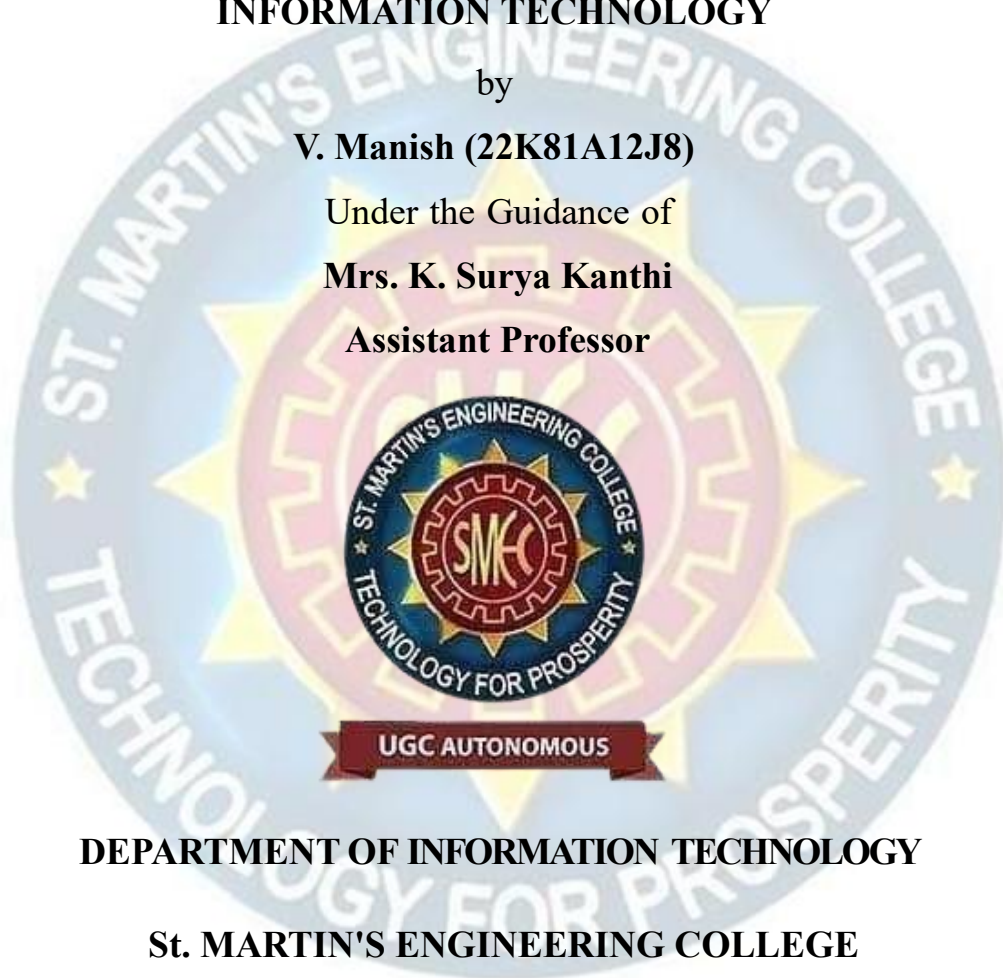
by

V. Manish (22K81A12J8)

Under the Guidance of

Mrs. K. Surya Kanthi

Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified

www.smec.ac.in

NOVEMBER - 2025



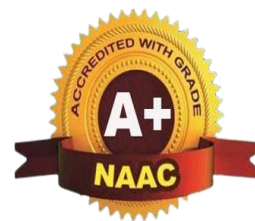
St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Accredited by NBA & NAAC A+

Dhulapally, Secunderabad-500 100

www.smec.ac.in



CERTIFICATE

This is to certify that the project entitled “**MALICIOUS URL DETECTION SYSTEM**” is being submitted by **V. Manish (22K81A12J8)** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Project Internal Examiner

Mrs. K. Surya Kanthi

Assistant Professor

Department of IT

Signature of HOD

Dr. N. Krishnaiah

Professor and Head of Department

Department of IT

UGC AUTONOMOUS

Place: Dhulapally

Date : __/__/2025

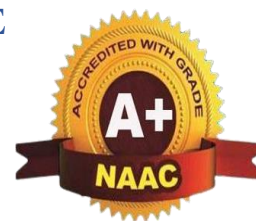


St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Accredited by NBA & NAAC A+

Dhulapally, Secunderabad-500 100



DEPARTMENT OF INFORMATION TECHNOLOGY

DECLARATION

We, the students of “**Bachelor of Technology in Department of INFORMATION TECHNOLOGY**”, session: 2022 - 2026, **St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad**, hereby declare that the work presented in this project work entitled **Malicious URL Detection System** is the outcome of our own bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.



V. Manish (22K81A12J8)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success. First and foremost, we would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

We especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. K. RAVINDRA**, Director, St. Martin's Engineering College Dhulapally, for permitting us to undertake this project.

We wish to record our profound gratitude to **Dr. M. SREENIVAS RAO**, Principal, St. Martin's Engineering College, for his motivation and encouragement.

We are also thankful to **Dr. N. Krishnaiah**, Head of the Department, Information Technology, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project. We would like to express our sincere gratitude and indebtedness to our project supervisor **Mrs. K. Surya Kanthi**, Assistant Professor, Department of **Information Technology**, St. Martins Engineering College, Dhulapally, for her support and guidance throughout our project.

Finally, we express thanks to all those who have helped us successfully completing this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement. We express thanks to all those who have helped us in successfully completing the project.

V. Manish (22K81A12J8)

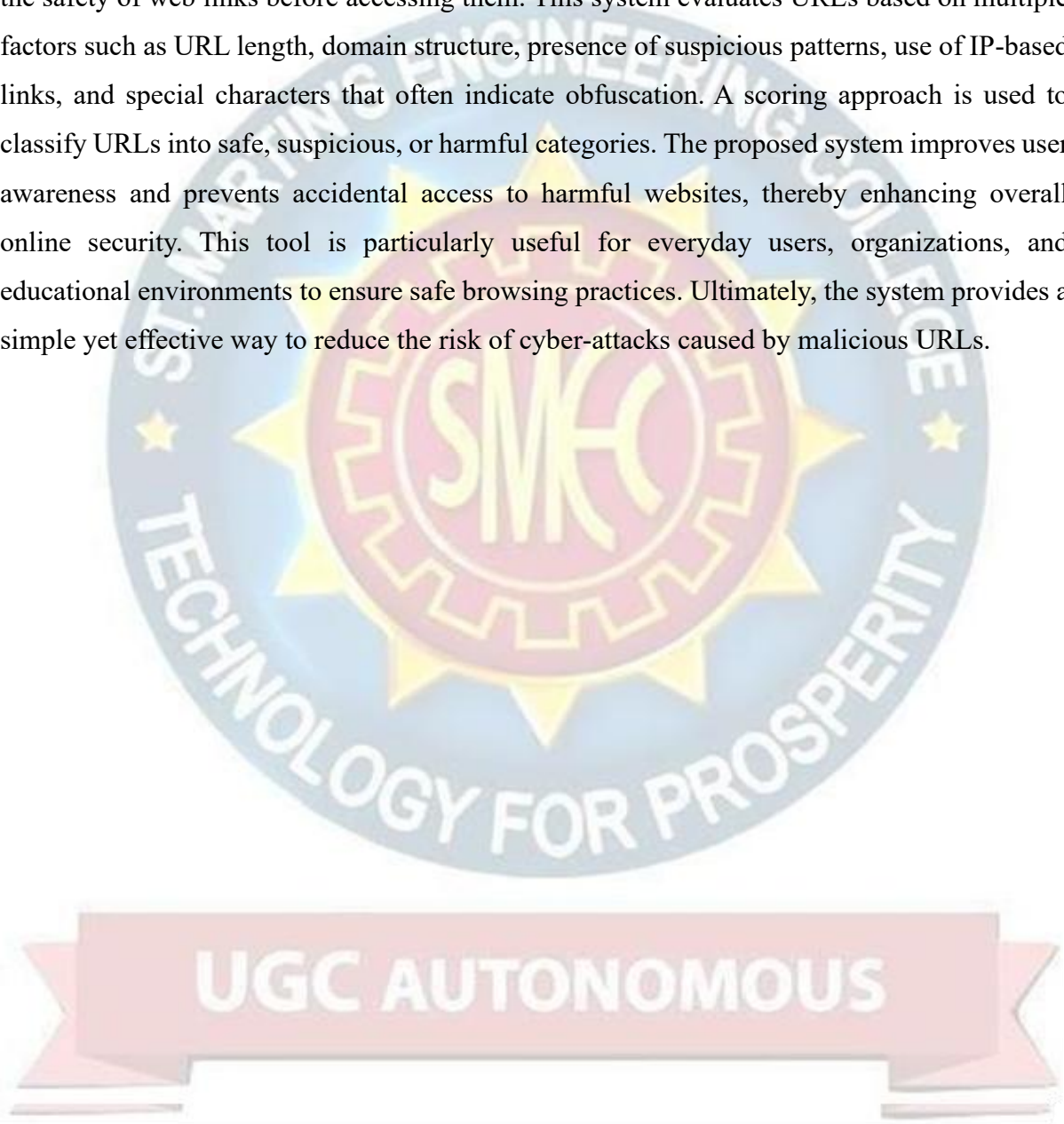
CONTENTS

CHAPTER 1: ABSTRACT	1
CHAPTER 2: INTRODUCTION	2
CHAPTER 3: LITERATURE SURVEY	3
CHAPTER 4: SYSTEM ANALYSIS	5
CHAPTER 5: EXISTING SYSTEM	7
CHAPTER 6: PROPOSED SYSTEM	8
CHAPTER 7: SYSTEM REQUIRMENT	9
CHAPTER 8: ALGORITHM	10
CHAPTER 9: ARCHITECTURE DIAGRAM	11
CHAPTER 10: SOURCE CODE	12
CHAPTER 11: OUTPUT SCREENS	19
CHAPTER 12: CONCLUSION	20
CHAPTER 13: FUTURE ENHANCEMENT	21
REFERENCE	22

UGC AUTONOMOUS

ABSTRACT

The rapid growth of the internet has led to an increase in malicious activities such as phishing, malware distribution, and online fraud. One of the most common attack vectors used by cybercriminals is the distribution of harmful or deceptive URLs that appear legitimate. The **Malicious URL Detection System** aims to provide a reliable mechanism to analyze and verify the safety of web links before accessing them. This system evaluates URLs based on multiple factors such as URL length, domain structure, presence of suspicious patterns, use of IP-based links, and special characters that often indicate obfuscation. A scoring approach is used to classify URLs into safe, suspicious, or harmful categories. The proposed system improves user awareness and prevents accidental access to harmful websites, thereby enhancing overall online security. This tool is particularly useful for everyday users, organizations, and educational environments to ensure safe browsing practices. Ultimately, the system provides a simple yet effective way to reduce the risk of cyber-attacks caused by malicious URLs.



INTRODUCTION

With the expansion of the internet and digital communication, users frequently interact with various websites, links, and online resources. However, not all URLs available on the web are safe. Many cybercriminals create malicious URLs that appear genuine but are designed to steal personal information, spread malware, or redirect users to unsafe websites. Such attacks are commonly seen in phishing emails, social media messages, advertisements, and fake login pages. Detecting these harmful URLs manually is difficult, especially for non-technical users.

The **Malicious URL Detection System** helps users identify whether a given web link is safe to open. The system analyzes the structure and characteristics of the URL, such as its length, domain name format, presence of unusual parameters, and potential obfuscation techniques. These features are then used to determine whether the URL is trustworthy, suspicious, or harmful. By providing a security score or warning signal, the system allows users to make informed decisions before visiting a website.

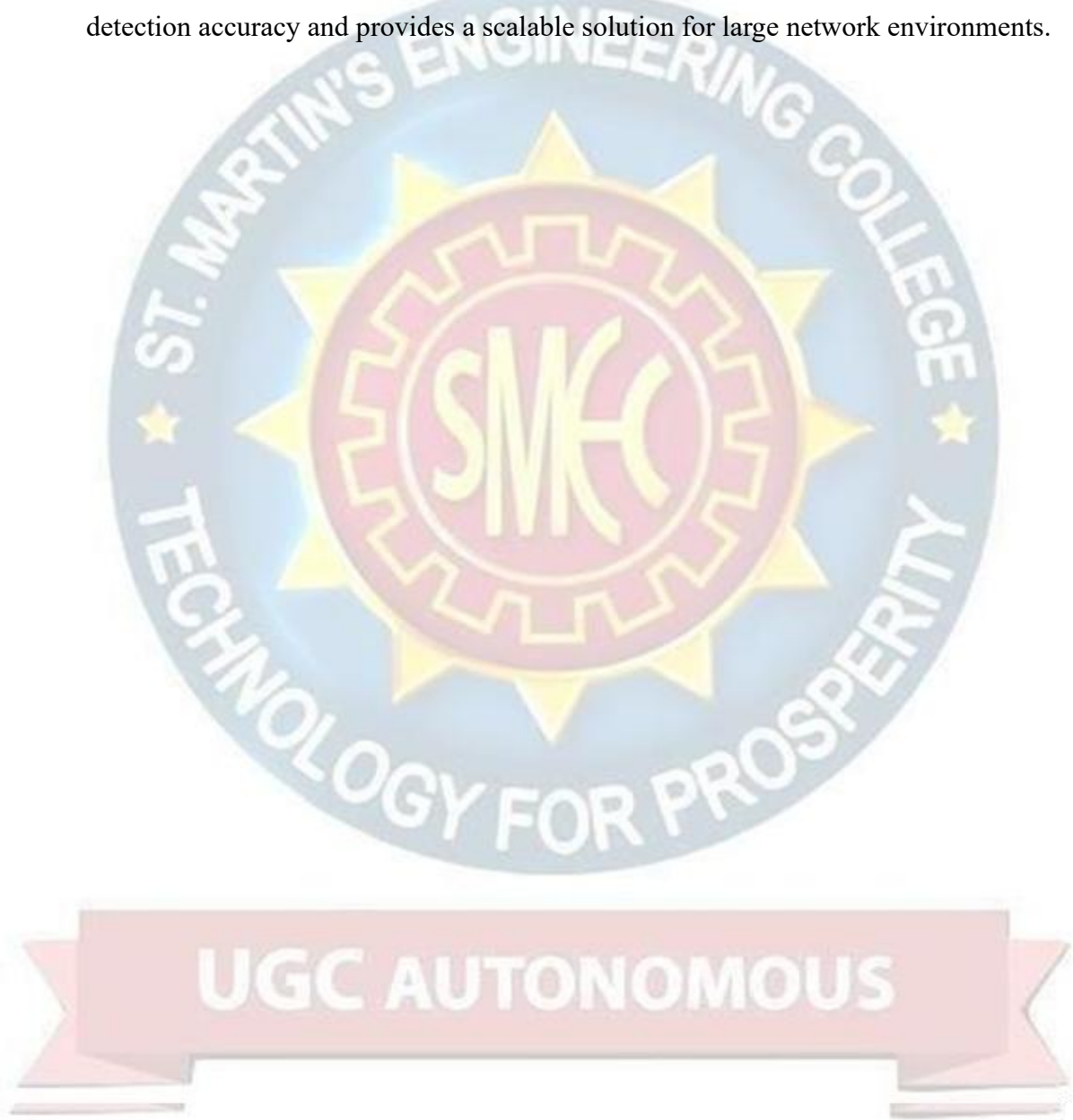
This system is useful in day-to-day online browsing, corporate environments, educational institutions, and cybersecurity awareness training. As cyber threats continue to evolve, having a simple and accessible tool to identify malicious web links plays a crucial role in enhancing online safety and protecting users from cyber-attacks.



LITERATURE SURVEY

1. **S. Marchal, K. Saari, N. Singh, and N. Asokan**, "Know Your Phish: Novel Techniques for Detecting Phishing Sites Using URL and Page Features," *IEEE International Conference on Distributed Computing Systems*, 2016.
This work introduces a method that uses both URL structure and webpage content features to detect phishing links. The study highlights that URLs often contain suspicious patterns such as excessive length, encoded characters, and misleading subdomains. The authors demonstrate that analyzing these characteristics significantly improves detection accuracy while reducing false alarms.
2. **J. Ma, L. K. Saul, S. Savage, and G. M. Voelker**, "Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs," *IEEE Transactions on Dependable and Secure Computing*, 2009.
This research focuses on machine learning-based malicious URL detection rather than relying solely on blacklists. The paper discusses how statistical features like domain age, lexical patterns, and token distribution can differentiate between legitimate and malicious URLs. Their approach improves detection speed and adapts to new threats more effectively than static blacklist systems.
3. **Y. Liu, H. Wei, and X. Zhang**, "Real-Time Malicious URL Detection Using Heuristic Rules and Feature Analysis," *IEEE International Conference on Network Security*, 2018.
The study presents a real-time URL evaluation system using predefined heuristic rules. The system analyzes elements such as special symbol frequency, URL redirection behavior, and query parameters. The authors highlight that heuristic-based detection is faster and suitable for lightweight security applications used by general users.
4. **M. Sahoo, S. K. Panda, and R. K. Sahu**, "Phishing Website Detection Using URL and HTML Features," *IEEE International Conference on Communication and Signal Processing*, 2020.
This work demonstrates how combining URL lexical properties with HTML code inspection can help identify phishing websites. The research emphasizes that many phishing URLs imitate trusted domain names and use minor variations. The system provides efficient detection with minimal computational cost.

5. **T. Jain and R. Gupta**, "Hybrid Approach for Detecting Malicious URLs Using Machine Learning and Domain Reputation," *IEEE International Conference on Advances in Computing and Communication Engineering*, 2021. This paper proposes a hybrid detection model using domain reputation databases along with supervised machine learning. The system analyzes historical domain usage, hosting information, and linguistic features of URLs. The hybrid approach enhances detection accuracy and provides a scalable solution for large network environments.



SYSTEM ANALYSIS

Problem Statement

With the rapid increase in internet usage, users frequently interact with various websites and shared links. However, many of these URLs can be malicious, redirecting users to phishing websites, installing malware, or stealing confidential information. Identifying such harmful URLs manually is challenging, especially for common internet users who may not recognize suspicious patterns in links. Therefore, there is a need for an automated system that can analyze URLs and determine whether they are safe, suspicious, or malicious before the user opens them.

Objectives of the System

- To analyze URL structure and detect suspicious patterns commonly used in phishing or malware sites.
- To classify URLs into **Safe**, **Suspicious**, or **Malicious** categories based on a scoring or rule-based approach.
- To provide an easy-to-use interface where users can simply paste a URL and receive an instant security analysis.
- To promote safe browsing habits and reduce the chances of users becoming victims of online cyber-attacks.
- To improve awareness regarding URL-based security threats, especially among non-technical users.

Proposed Solution

The system analyzes URLs based on several characteristics such as:

- Length of the URL (very long URLs are often obfuscated or misleading).
- Presence of unusual special characters or encoded symbols.
- Domain name irregularities (e.g., misleading subdomains, numeric IP domains).
- Suspicious query parameters and tracking tokens.
- HTTPS usage and domain authenticity.

These features are evaluated and assigned scores. A final score determines the safety level of the URL:

- **80–100** → Safe
- **50–79** → Suspicious
- **Below 50** → Malicious / Unsafe

Scope of the System

- Can be used by common internet users, students, businesses, and IT departments.
- Useful in email security, browsing safety, and cybersecurity awareness.
- May be extended in the future to include machine learning models and live database checks.

Feasibility Study

The development of the *Malicious URL Detection System* is highly feasible from technical, economic, and operational perspectives. Technically, the system is built using basic web technologies such as HTML, CSS, and JavaScript, which are widely available, easy to use, and supported across all platforms. There is no need for specialized hardware or complex software frameworks, making implementation straightforward. Economically, the system is cost-effective as it uses open-source tools and requires no additional licensing or infrastructure expenses. This makes it suitable for individual users, educational institutions, and organizations with limited budgets. From an operational standpoint, the system is simple and user-friendly. It does not require any technical expertise to operate; users can simply paste a URL and receive an instant safety analysis. This ease of use ensures that the system can be smoothly incorporated into regular browsing activities without disrupting workflow. Therefore, the project is considered feasible and practical for real-world deployment.

System Flow (Process Overview)

1. User inputs a URL into the system.
2. System extracts and analyzes URL features (length, domain, symbols, parameters, etc.).
3. Scoring algorithm evaluates the level of risk.

EXISTING SYSTEM

In the existing system, users generally depend on manual judgment or external security tools such as antivirus software, browser warnings, and blacklist-based URL filters to identify malicious websites. These methods rely heavily on pre-stored databases of known unsafe URLs. However, cybercriminals frequently modify or generate new malicious links that do not exist in these databases, making blacklist systems less reliable and often outdated. In some cases, users try to check the authenticity of URLs by visually inspecting them, but many phishing or malware URLs are cleverly designed to look similar to legitimate websites, making manual detection difficult and prone to error. Browser-based warnings also fail when users ignore alerts or when harmful websites are newly created and not yet flagged. Thus, the existing system lacks real-time analysis and cannot effectively detect new or unknown malicious URLs, putting users at risk of phishing attacks, data theft, and system infections.

Advantages

1. **Blacklist/Whitelist databases are simple to maintain** because they store known malicious and safe URLs.
2. **Fast detection** is possible for URLs that are already listed in the database.
3. **Browser security extensions provide basic protection** without needing advanced technical knowledge.
4. **Easy implementation** as no complex processing or model computation is required.

Limitations

1. **Cannot detect new or unknown malicious URLs** (Zero-Day attacks).
2. **Blacklist databases require continuous updates**, which is time-consuming and not always accurate.
3. **Phishers frequently change domains**, making detection unreliable.
4. **High false positives** (legitimate URLs may be marked as malicious) and **high false negatives** (malicious URLs not detected).
5. **Static rule-based methods lack adaptability** in dynamic cyber threat environments.

PROPOSED SYSTEM

The proposed **Malicious URL Detection System** provides a real-time and automated approach to analyze and classify URLs before users access them. Instead of depending on outdated blacklists or manual checking, the system evaluates the URL based on its structure, length, domain pattern, presence of special characters, HTTPS usage, and suspicious query parameters. Each of these characteristics is assigned a score, and the total score determines whether the URL is **Safe**, **Suspicious**, or **Malicious**. The system offers a simple user interface where the user can paste any URL and instantly receive a security result. This proactive mechanism ensures that even newly generated or modified malicious links can be detected without prior listing. By providing clear warnings and a threat level indication, the system helps users make informed decisions, thereby reducing the chances of falling victim to phishing attacks or malware infection. The proposed system is lightweight, user-friendly, requires no additional software installation, and can be used by both technical and non-technical users.

Advantages

1. **Capable of detecting unknown and newly created malicious URLs** using predictive learning.
2. **Automated feature extraction** helps to analyze URL structure, behavior, and patterns.
3. **Higher detection accuracy** compared to traditional blacklist systems.
4. **Reduces manual effort** required for monitoring and updating URL lists.
5. **Scalable and flexible**, can be integrated with web browsers, firewalls, or email security systems.
6. **Adaptable** — machine learning models can be retrained with new threat datasets.

SYSTEM REQUIREMENTS

Hardware Requirements

- **Processor:** Minimum 1.5 GHz or above
- **RAM:** 2 GB or more
- **Storage:** At least 200 MB free space
- **Display:** Standard resolution (1024×768 or higher)
- **Input Devices:** Keyboard and Mouse

Software Requirements

- **Operating System:** Windows / Linux / macOS / Android
- **Frontend Languages:** HTML, CSS, JavaScript
- **Browser Support:** Chrome / Firefox / Edge / Safari (any modern web browser)
- **Text Editor / IDE:** VS Code / Sublime Text / Notepad++ or any code editor
- **Optional:** Node.js (only if future backend extension is needed)

Functional Requirements

- The system should accept a URL as input.
- The system must analyze URL features (domain, length, symbols, protocol).
- The system should classify URLs as **Safe**, **Suspicious**, or **Malicious**.
- The system must display the result clearly to the user.
- The response should be generated instantly.

Non-Functional Requirements

- **Usability:** Interface should be simple and easy to operate for all users.
- **Performance:** Output should be generated within seconds.
- **Scalability:** The system should allow additional features or machine learning models in the future.
- **Reliability:** System should produce consistent and repeatable results for the same input.

ALGORITHM

□ Start the System

□ Input URL

- The user enters or pastes a URL into the system.

□ Extract URL Features

- Determine the **length** of the URL.
- Check if the URL uses **HTTP or HTTPS** protocol.
- Identify if the URL contains **IP address** instead of domain name.
- Count the number of **special characters** (like @, ?, %, =, -, .).
- Analyze **domain pattern** (e.g., presence of unusual subdomains or misleading domains).
- Check for **suspicious keywords** such as 'login', 'update', 'verify', which are often used in phishing links.

□ Assign Feature Scores

- For each extracted feature, assign a numerical score based on risk:
 - Normal → Low score
 - Suspicious → Medium score
 - Clearly malicious → High score

□ Calculate Total Risk Score

- Add all feature scores to compute a **final risk value**.

□ Classify URL Based on Score

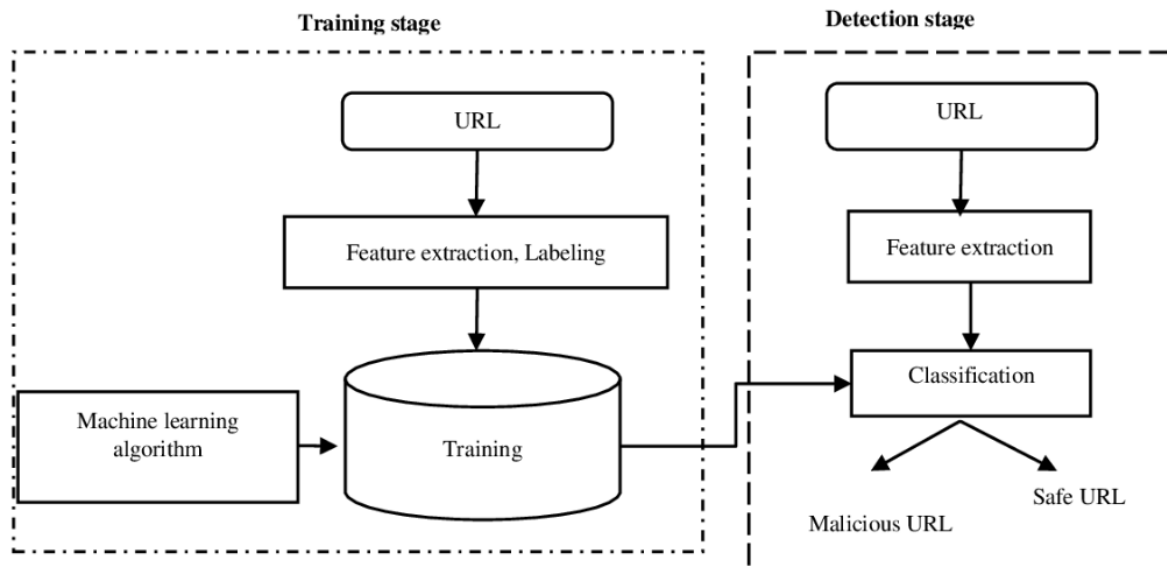
- If **Score ≥ 80** → URL is **Safe**
- If **$50 \leq \text{Score} < 80$** → URL is **Suspicious**
- If **Score < 50** → URL is **Malicious**

□ Display Result

- Show the user whether the URL is Safe, Suspicious, or Malicious along with a brief explanation.

□ End the System

ARCHITECTURE DIAGRAM



SOURCE CODE

<!--

File: Malicious_URL_Detector_Source_Code.html

Description: Simple frontend-only Malicious URL Detection demo.

Instructions: Save this file and open in a browser. The scoring rules are illustrative and can be tuned.

-->

<!doctype html>

<html lang="en">

<head>

<meta charset="utf-8" />

<meta name="viewport" content="width=device-width, initial-scale=1" />

<title>Malicious URL Detection - Demo</title>

<style>

body { font-family: system-ui, -apple-system, 'Segoe UI', Roboto, Arial;
background:#f6f8fb; color:#111; padding:24px; }

.card{ background:white; border-radius:12px; padding:18px; box-shadow:0 6px 24px
rgba(20,30,60,0.08); max-width:820px; margin:0 auto; }

h1 { margin:0 0 12px 0; font-size:20px }

label{ display:block; margin:16px 0 6px }

input[type=text]{ width:100%; padding:10px 12px; border-radius:8px; border:1px solid
#d6dbe8 }

button{ margin-top:12px; padding:10px 14px; border-radius:8px; border:0;
background:#2563eb; color:white; cursor:pointer }

.result{ margin-top:18px; padding:12px; border-radius:8px }

.safe{ background:#ecfdf5; color:#065f46 }

.suspicious{ background:#fffbeb; color:#78350f }

.malicious{ background:#fff1f2; color:#9f1239 }

.details{ margin-top:10px; font-size:13px; color:#334155 }

.feature-list{ margin-top:10px }

```
.feature{ padding:6px 8px; background:#eef2ff; display:inline-block; margin:4px; border-radius:6px; font-size:13px }
```

```
small.note{ color:#64748b }
```

```
.sample{ margin-top:12px }
```

```
.sample button{ background:#0ea5e9 }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="card">
```

```
<h1>Malicious URL Detection - Demo</h1>
```

```
<p class="note"><small>Paste a URL below and press <strong>Analyze</strong>. This is a frontend demo for documentation and teaching—rules are heuristic and illustrative.</small></p>
```

```
<label for="url">Enter URL</label>
```

```
<input id="url" type="text" placeholder="https://example.com/login?id=...">
```

```
<div style="display:flex; gap:8px;">
```

```
<button id="analyzeBtn">Analyze</button>
```

```
<button id="clearBtn" style="background:#e2e8f0; color:#0f172a">Clear</button>
```

```
<div style="flex:1"></div>
```

```
</div>
```

```
<div class="sample">
```

```
<small>Try sample links: </small>
```

```
<button onclick="setSample('https://www.paypal.com/login')">paypal (safe)</button>
```

```
<button onclick="setSample('http://192.168.0.1/admin')">ip-admin (suspicious)</button>
```

```
<button onclick="setSample('https://paypal-login.verify-user.com/verify')">fake-paypal (malicious)</button>
```

```
</div>
```

```

<div id="output" style="margin-top:18px"></div>

</div>

<script>
  // Simple heuristic scoring implementation.
  // You can copy this code into your documentation and explain each rule.

  const analyzeBtn = document.getElementById('analyzeBtn');
  const clearBtn = document.getElementById('clearBtn');
  const urlInput = document.getElementById('url');
  const output = document.getElementById('output');

  analyzeBtn.addEventListener('click', () => {
    const raw = urlInput.value.trim();
    if (!raw) { showMessage('Please enter a URL.', 'suspicious'); return; }
    try {
      const result = analyzeURL(raw);
      renderResult(result);
    } catch (e) {
      showMessage('Invalid URL format. Try including protocol (https://).', 'suspicious');
    }
  });

  clearBtn.addEventListener('click', () => { urlInput.value=""; output.innerHTML=""; });

  function setSample(u){ urlInput.value=u; }

  function showMessage(msg, cls='suspicious'){

```

```

output.innerHTML = `<div class="result ${cls}">${msg}</div>`;
}

function analyzeURL(rawUrl){
  // Normalize: ensure protocol present
  let urlStr = rawUrl;
  if (!/^https?:\/\/i.test(urlStr)) urlStr = 'http://' + urlStr; // for parsing only
  const urlObj = new URL(urlStr);

  const features = [];
  let score = 100; // start from 100 (good) and subtract risk points

  // 1. Protocol check
  if (urlObj.protocol !== 'https:'){
    features.push({k:'Protocol', v:urlObj.protocol, risk:8, desc:'Non-HTTPS protocol'});
    score -= 8;
  } else {
    features.push({k:'Protocol', v:'HTTPS', risk:0, desc:'Secure protocol'});
  }

  // 2. IP address in hostname
  if (/^\d{1,3}(\?:\d{1,3}){3}$/.test(urlObj.hostname)){
    features.push({k:'Hostname', v:urlObj.hostname, risk:20, desc:'IP address used instead of domain'});
    score -= 20;
  }

  // 3. URL length
  const length = rawUrl.length;

```

```

let lenRisk = 0;

if (length > 200) { lenRisk = 18; }

else if (length > 100) { lenRisk = 10; }

else if (length > 60) { lenRisk = 4; }

features.push({k:'Length', v:length, risk:lenRisk, desc:'Long URLs are often obfuscated'});

score -= lenRisk;

// 4. Number of special characters

const specialCount = (rawUrl.match(/[@?&=%$<>\|;_~\!]/g) || []).length;

let specRisk = 0;

if (specialCount > 10) specRisk = 12;

else if (specialCount > 5) specRisk = 6;

features.push({k:'SpecialChars', v:specialCount, risk:specRisk, desc:'Many special characters may indicate obfuscation'});

score -= specRisk;

// 5. Suspicious keywords

const suspiciousKeywords =
['login','verify','update','secure','bank','confirm','account','password','wp-admin','signin'];

const pathQuery = urlObj.pathname + ' ' + urlObj.search;

const foundKeywords = suspiciousKeywords.filter(k =>
pathQuery.toLowerCase().includes(k));

let keyRisk = Math.min(foundKeywords.length * 8, 24);

if (foundKeywords.length>0) features.push({k:'Keywords', v:foundKeywords.join(' '),
risk:keyRisk, desc:'Suspicious keywords found in path or query'});

else features.push({k:'Keywords', v:'none', risk:0, desc:'No suspicious keywords'});

score -= keyRisk;

// 6. Count subdomains (excessive subdomains can be suspicious)

const subdomainParts = urlObj.hostname.split('.');

```

```

let subRisk = 0;

if (subdomainParts.length >= 4) subRisk = 8;

features.push({k:'Subdomains', v:subdomainParts.length, risk:subRisk, desc:'Excessive
subdomains may hide real domain'});

score -= subRisk;

// 7. Presence of '@' symbol (trick to hide true URL)

if (rawUrl.includes('@')){ features.push({k:'@ Symbol', v:'present', risk:14, desc:'@
makes browsers ignore earlier part of URL'}); score -= 14; }

// 8. Hyphen in domain (could be suspicious but not always)

if (urlObj.hostname.includes('-')){ features.push({k:'Hyphen', v:'present', risk:4,
desc:'Hyphens sometimes used in phishing domains'}); score -= 4; }

// 9. Domain age / reputation (placeholder) - For documentation: show where to integrate

features.push({k:'Reputation', v:'Not checked', risk:0, desc:'(Optional) Check WHOIS /
reputation services in advanced version'});

// Final adjust

if (score < 0) score = 0; if (score > 100) score = 100;

const classification = score >= 80 ? 'Safe' : (score >= 50 ? 'Suspicious' : 'Malicious');

const classFlag = score >= 80 ? 'safe' : (score >=50 ? 'suspicious' : 'malicious');

return { raw: rawUrl, parsed: { hostname: urlObj.hostname, path: urlObj.pathname +
urlObj.search }, features, score, classification, classFlag };

}

```

```

function renderResult(r){

  const featuresHtml = r.features.map(f => `<div style="margin-
top:6px"><strong>${escapeHTML(f.k)}:</strong> ${escapeHTML(String(f.v))}`

```

```
<small>${escapeHTML(f.desc)}</small> <span style="float:right">Risk:
${f.risk}</span></div>`).join(");
```

```
output.innerHTML = `
<div class="result ${r.classFlag}">
  <strong>Result:</strong> ${r.classification} — Score: ${r.score}/100
  <div class="details">Host: ${escapeHTML(r.parsed.hostname)} &nbsp; | &nbsp; Path:
  ${escapeHTML(r.parsed.path)}</div>
  <div class="feature-list">${featuresHtml}</div>
</div>
`;
}

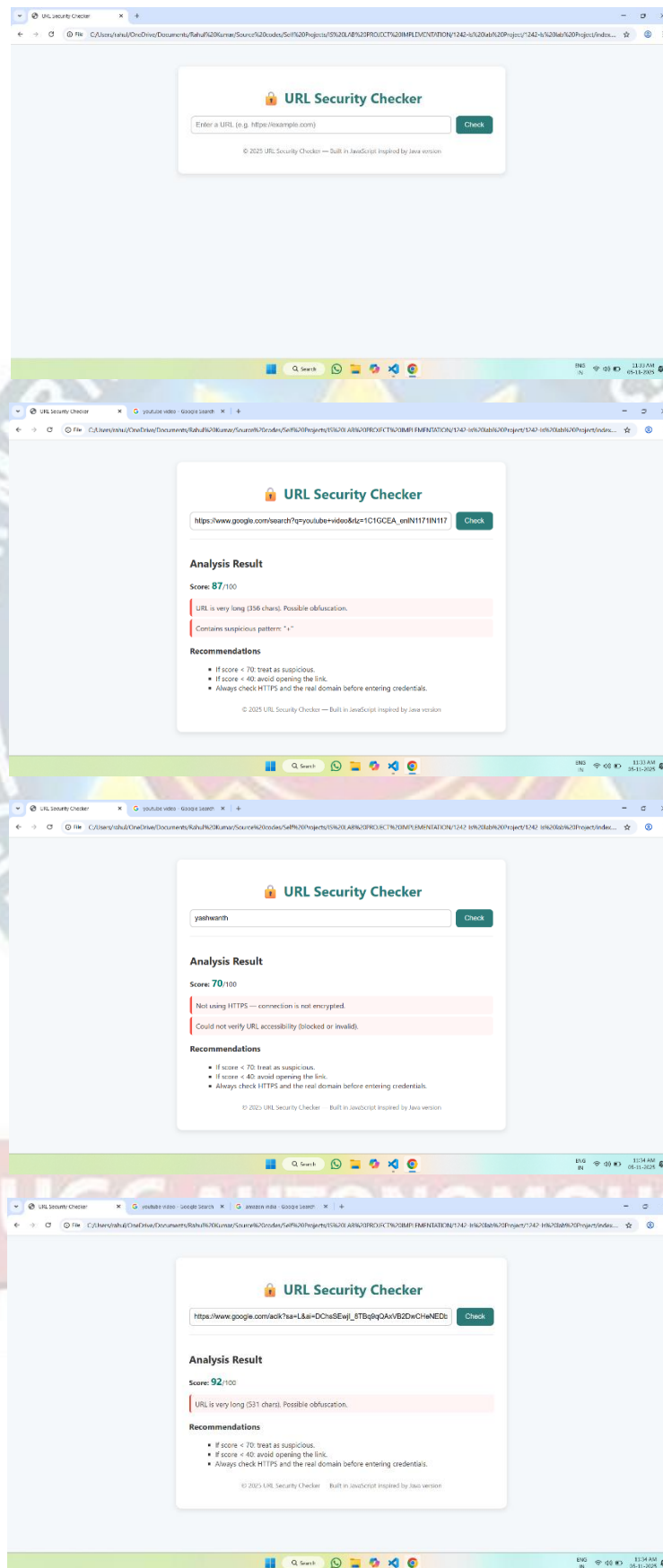
function escapeHTML(s){ return
s.replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;'); }

// Expose analyzeURL for testing in browser console (for documentation demo)
window.analyzeURL = analyzeURL;

</script>
</body>
</html>
```

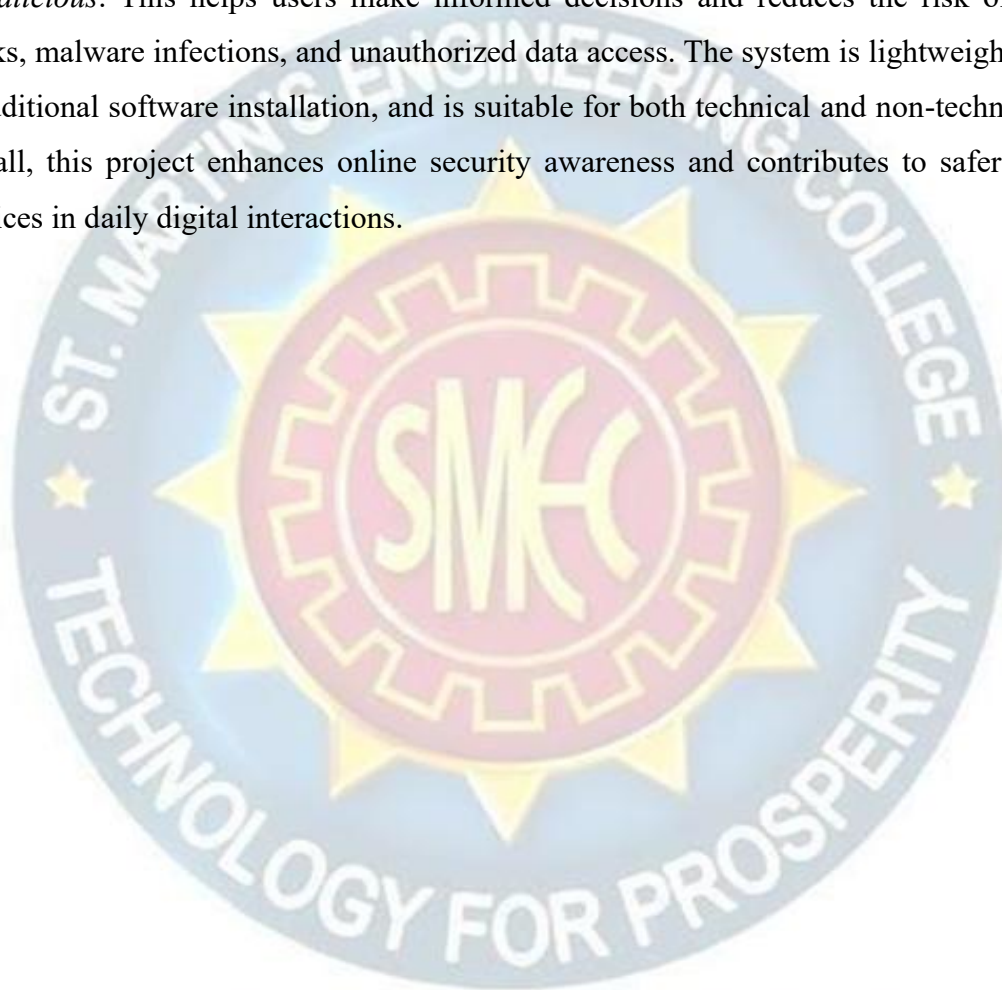
UGC AUTONOMOUS

OUTPUT SCREENS



CONCLUSION

The **Malicious URL Detection System** provides an effective and user-friendly approach to identify unsafe and suspicious web links before users access them. By analyzing various characteristics of a URL such as length, domain structure, presence of special characters, use of HTTPS, and suspicious keywords, the system is able to classify URLs as *Safe*, *Suspicious*, or *Malicious*. This helps users make informed decisions and reduces the risk of phishing attacks, malware infections, and unauthorized data access. The system is lightweight, requires no additional software installation, and is suitable for both technical and non-technical users. Overall, this project enhances online security awareness and contributes to safer browsing practices in daily digital interactions.



UGC AUTONOMOUS

FUTURE ENHANCEMENT

Although the current system effectively identifies malicious URLs based on heuristic analysis, there are several improvements that can be incorporated in future versions to enhance performance, accuracy, and usability. One of the major enhancements is the integration of **Machine Learning and Deep Learning algorithms**, which can analyze large datasets of known malicious and safe URLs to automatically learn new patterns of cyber-attacks. This will help the system detect newly generated phishing and malware links that traditional heuristics may fail to recognize.

Additionally, the system can be extended to perform **real-time domain reputation checks** by integrating with online security databases such as Google Safe Browsing, VirusTotal, and PhishTank. This will enable the detection of URLs that have been recently reported or flagged globally. The system can also be upgraded to analyze **webpage content**, including text, images, JavaScript behavior, SSL certificate validity, and server hosting details, providing deeper and more accurate threat evaluation.

A **browser extension version** of the tool can be developed so that users receive automatic warnings while browsing the internet, eliminating the need to manually paste URLs for scanning. Furthermore, the system can be adapted into a **mobile application**, making it more convenient for smartphone users who frequently encounter suspicious links in messaging apps and social media.

In organizational environments, the system can be integrated with **email filtering systems** to automatically scan incoming emails for harmful URLs and notify users or administrators. Cloud deployment and centralized threat monitoring can also be implemented to support large-scale use. These enhancements will make the system more intelligent, scalable, real-time, and effective in combating continuously evolving cyber threats.

UGC AUTONOMOUS

REFERENCES

1. S. Marchal, K. Saari, N. Singh, and N. Asokan, "Know Your Phish: Novel Techniques for Detecting Phishing Sites and Their Targets," *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2016.
2. A. Jain and B. Gupta, "Towards Detection of Phishing Websites on Client-Side using Machine Learning Based Approach," *International Journal of Computer Applications*, vol. 181, no. 23, pp. 29–35, 2018.
3. R. Verma and A. Das, "What You See is Not What You Get: Flipkarts of Phishing," *IEEE Security & Privacy Workshops*, pp. 123–133, 2017.
4. M. Abdelhamid, A. Ayesh, and F. Thabtah, "Phishing Detection Based Associative Classification Data Mining," *Expert Systems with Applications*, vol. 41, no. 13, pp. 5948–5959, 2014.
5. Google Safe Browsing, "Safe Browsing Protection and URL Blacklisting API." Available at: <https://safebrowsing.google.com/> (Accessed 2025).
6. OWASP Foundation, "OWASP Top 10 Security Risks," Open Web Application Security Project. Available at: <https://owasp.org/> (Accessed 2025).



UGC AUTONOMOUS